



UNIVERSITY PARTNER



Artificial Intelligence and Machine Learning — 6CS012

Question Report

Q & A

Submitted By : Dipesh Sherchan

Student Id : 2438455

Group : L6CG7

Submission Date : 2026/05/10

Long Question 1

Q. You are a Machine Learning Engineer at a rapidly growing e-commerce company responsible for deploying and maintaining ML systems in production.

- Identify and explain at least three real-world challenges encountered in ML systems.
- For each challenge, discuss consequences, propose practical solutions, highlight trade-offs.
- Reflect on cross-functional collaboration and comment on how modern tools may help.

Answer:

Introduction

Machine learning systems are moving from a lab environment to production environments, and they are navigating new challenges in real-world applications. Offline experiments involve data that doesn't change over time and evaluation that happens in a predictable manner while production ML systems must deal with data that continuously changes, unpredictable traffic, changing user behavior, and strict reliability requirements. The three challenges below are all real-world challenges that will have to be solved by every ML engineering team and their implications, solutions, and trade-offs are included, as well as the importance of collaboration and modern tooling.

Challenge 1: Data Drift

- *What is Data Drift?*

Data drift is the situation in which the statistics of the data fed into a model in production change over time from the data used for training the model. This is a very usual and hazardous issue in a quickly growing e-commerce business. Suppose that a product recommendation model is trained on user behaviour information during a holiday season. After the season is over, consumers' shopping habits will vary dramatically — their search for products may differ, product categories may be new, and purchasing frequencies will change. The model, however, makes recommendations using old learned patterns and therefore becomes more and more irrelevant.

- *What are the consequences if not addressed?*

Consequences of data drift can be serious if it goes undetected and uncorrected in a timely manner. While the errors are not raised in an obvious manner, the greater the error in the model, the poorer the output of the system. For e-commerce, it means less click-through and less conversion rate, less satisfaction for investors, and lost revenue. The worst part of data drift is that it typically happens incrementally over time; there's no one moment it fails, and without monitoring systems specifically designed to detect data drift, it's easy to miss the evidence.

- *What are the real-world solutions?*

The best way is to construct a continuous data monitoring pipeline that is integrated with the production model. Comparing the distribution of incoming production data to a reference baseline from the training data can be achieved using tools like Evidently AI, WhyLogs or custom statistics tests such as Kolmogorov-Smirnov test. If the drift is greater than some limit, alerts are triggered and automatically sent to the engineering team for investigation. What is important is that the solution not only detects the new data, but also provides a response mechanism, such as an automated retraining pipeline that fetches fresh data, retrains the model, performs validation, and deploys the new model, while requiring as much as possible human involvement. Apache Airflow or Kubeflow Pipelines are frequently used to manage these workflows. Also, it is important to keep a set of recent production data in a sliding window for monitoring purposes to keep the reference distribution relevant over time.

- *What are the drawbacks to this?*

The biggest drawback of this solution is its price and complexity. Large deep learning models need a lot of compute resources to be trained automatically frequently. There is also risk associated with each retraining cycle: if the data set used to train the new model is noisy or unrepresentative, the new model may be less effective than the existing production model. This requires well-tested A/B testing and shadow deployment methods to be implemented before a complete switch to the new model. Teams also need to allocate time to establish suitable drift thresholds, thresholds that are too high will result in failure to detect drift, and thresholds that are too low will lead to too many false alarms.

Challenge 2: Model Degradation and Monitoring

- *Model Degradation – What is it?*

Data drift is just one type of model degradation — the degradation of a model's real-world performance over time for any reason. In the context of e-commerce, a fraud detection model can over time become less effective if the fraudster adapts and finds more and more patterns that the model is not “trained” to detect. In the same way, even if the model is trained on a few years of historical data, an abrupt change in macroeconomic factors, competitor pricing, or the supply chain may result in the model making progressively worse predictions. Model degradation is more subtle and gradual in comparison to a production bug, which results in a clear error to be fixed.

- *If left unattended, what will be the consequences?*

If the model is degrading but it continues to power the predictions, then it may continue to do so for extended periods of time, perhaps weeks or even months, before anyone realizes it's degraded. In fraud detection this equates to the ability to detect fraudulent transactions, resulting in direct financial losses and potentially regulatory fines. In a recommendation system, users start to see less relevant recommendation, thereby losing interest and churning out of the system. There's also a reputation element — when stakeholders and customers can see AI-powered features fail to deliver as expected,

trust in the technology is lost and it becomes harder to make a business case for future investment in ML projects.

- *What are the solutions that can be implemented to improve practice?*

The key to tackling model degradation is a complete real-time monitoring system. This system should include not only the technical metrics like prediction latency and throughput, but other model-specific metrics like prediction confidence score distributions, output class balance, and, if ground truth labels are available, live accuracy, precision, recall, and F1 scores as well. Experiment tracking and model versioning is enabled with tools such as MLflow, and model health is real-time visualised with Grafana dashboards linked to Prometheus metrics collectors. Another powerful technique is shadow deployments: a new candidate model is run concurrently with the production model, and both models are given the same inputs, but the model's predictions are not shown to the users. This enables the team to test the behavior of the new model and hit the production model before committing to rollout. Enabling Canary deployments, in which the new model is deployed to an increasing fraction of traffic, further minimizes the impact of the bad model on most traffic.

- *What are the compromises?*

Operational overhead is added by comprehensive monitoring. The monitoring infrastructure must be designed, constructed, and maintained by engineers, meaningful alert thresholds must be set, and alerts must be acted on quickly. Alert fatigue is a potential threat — too aggressive and the on call team gets too many alerts for small fluctuations that will work out on their own. This can result in significant notifications being ignored or de-prioritised. To monitor effectively, without becoming an onerous task, careful calibration of thresholds, grouping of alerts and a clear escalation path are required.

Challenge 3: Scalability and Latency

- *What is the Scalability and Latency challenge?*

The more an e-commerce platform expands, the more it needs to support millions of users concurrently, and it can't afford to have a prediction latency that varies above a given threshold. As an e-commerce platform scales up, its ML systems must scale up to support millions of users concurrently and with a prediction latency that always stays below a threshold. This is fine for a development product with a response time of 500 ms, but in production a user will expect a page to load quickly – within 100 ms – otherwise they will feel like they are waiting too long. This is a real challenge for applications which require real-time predictions like personalized search ranking, dynamic pricing, fraud detection at the checkout and live inventory recommendations, where predictions must be delivered in a time budget of 50-100 milliseconds or less.

- *What are the consequences if not addressed?*

High latency directly impacts on user experience. Studies have found that a mere delay of one second in page load time can cause a 7% drop in conversion rate, a 103% rise in bounce rates, and a 100% increase in social sharing. In fact, studies have confirmed that a one-second delay in page load time results in an 8% decrease in conversions, a 103% increase in bounce rates, and a 100% increase in social sharing. In addition to user experience, failures during high traffic may lead to cascading failures in the whole system; if the ML serving layer is a slowdown of the whole platform, the problem will not just impact on the recommendation or ranking system, but on the entire system. In situations that require critical business function, like during peak traffic events like flash sales or major holidays, an unscalable ML system can be the cause of the entire system going offline.

- *What are the "real" solutions?*

There are multiple complementary approaches to tackle scalability and latency issues. The memory and inference time can be significantly decreased by quantization, i.e., decrease the number of bits used to store the model parameters, pruning, i.e., remove redundant model parameters, and knowledge distillation, i.e., train a smaller model (called student) to mimic a larger model (called teacher). The benefit of prediction caching is in situations when the same input is used repeatedly. Containerizing models with Docker and using Kubernetes to orchestrate the system allows for horizontal scaling, which means that the system can automatically start new serving instances when traffic rates increase. For production ML deployments, use a dedicated model serving framework like TensorFlow Serving, TorchServe, or NVIDIA Triton Inference Server instead of general purpose web servers, which are designed for serving models in a high throughput, low latency manner.

- *How does this affect the other angles of the triangle?*

In optimizing for scalability, the trade off is between speed and accuracy. When it comes to high-stakes tasks, such as fraud detection, quantization and pruning can cause a slight loss in model accuracy that can be detrimental. Only repetitive inputs are good for caching; if cached predictions are not invalidated when the underlying model changes, there is a risk of staleness. Horizontal scaling is slower and more expensive, but more scalable when load increases, which necessitates a capacity plan and cost-benefit analysis.

- ***What is the role of cross-functional collaboration in ensuring reliable ML systems?***

Creating and maintaining a trustworthy ML system in production is a team game. Data scientists possess knowledge and skills in model design, feature engineering and evaluation of model performance. ML engineers spend their time on the deployment infrastructure that is scalable, reliable, and easy to maintain. Product managers give the business background that defines acceptable performance, failure, and how to present the capabilities of the model to the users. Closing the loop on these functions is essential, as otherwise technically perfect models often fail in production, either due to the lack of infrastructure that is able to support them at scale, or because they are optimized for the wrong metrics

altogether. It's important to have regular cross-functional syncs, shared success metrics and dashboards that everyone can refer to to ensure all teams are aligned.

- *What are the role and benefits of modern tools in solving these problems?*

Today's MLOps platforms like Google Vertex AI, AWS SageMaker, and Azure ML offer full pipeline automation for training, validation, deployment and monitoring, significantly cutting down manual effort. New LLM-based systems are now starting to help automate anomaly detection in pipelines, intelligent alert triage and even automated self-healing retraining workflows. The tools enable engineering teams to steer their attention away from repetitive operations and onto more valuable activities like system architecture, experimentation strategy and business alignment, all of which help to increase iteration cycles and better future-proof production ML systems.

Short Question 1 – Overfitting Techniques

Q. Overfitting is a common challenge in deep learning, especially when models have high capacity. Describe at least two techniques used to reduce overfitting. For each technique, explain the intuition, how it affects training, and provide a real-world application example.

Answer:

Overfitting occurs when a deep learning model learns the training data too well — including its noise and irrelevant patterns — resulting in poor generalization to unseen data. Two widely used techniques to combat overfitting are Dropout and Early Stopping.

- **Technique 1: Dropout**

Dropout works by randomly deactivating a percentage of neurons during each training step. The intuition behind dropout is that by preventing neurons from co-adapting too strongly to specific training examples, the network is forced to learn more robust and distributed representations of the data. During training, this acts as an implicit ensemble of many different network architectures. During inference, all neurons are active but their outputs are scaled accordingly. In practice, dropout significantly reduces overfitting in high-capacity networks. A real-world example is image classification — when training a deep CNN to classify medical images such as X-rays, dropout prevents the model from memorizing specific pixel patterns in the training set and instead encourages it to learn clinically meaningful features that generalize to new patient scans.

- **Technique 2: Early Stopping**

Early stopping monitors the model's performance on a validation set during training and halts the training process as soon as validation loss stops improving, even if training loss continues to decrease. The intuition is straightforward — the point at which validation loss begins rising while training loss keeps falling is precisely where the model transitions from learning generalizable patterns to memorizing training-specific noise. By stopping at this optimal point, we preserve the best version of the model before overfitting sets in. A real-world example is text classification for fraud detection — training an LSTM model on transaction descriptions where early stopping prevents the model from over-specializing on historical fraud patterns that may not reflect future fraud behavior, thereby maintaining better generalization on new unseen transactions.

Conclusion

Both dropout and early stopping are complementary regularization strategies. Dropout introduces structured randomness during training to encourage robust feature learning, while early stopping controls the duration of training to prevent the model from over-optimizing on training data. Together, they form a strong defense against overfitting in deep learning systems across domains such as computer vision, natural language processing, and financial modeling.

Short Question 2 – FCN vs Autoencoder

Q. Compare a standard feedforward neural network (fully connected network) with an autoencoder in terms of architecture structure, objective/functionality, and input-output relationship. Provide a simple example, describe at least one application of autoencoders, and explain how autoencoders help solve the chosen problem.

Answer:

Architecture Structure

A standard Fully Connected Network (FCN) consists of an input layer, one or more hidden layers, and an output layer where every neuron in one layer is connected to every neuron in the next. Each layer progressively transforms the input toward a final prediction. An autoencoder, on the other hand, has a distinctly symmetric architecture divided into two parts — an encoder and a decoder. The encoder compresses the input data into a lower-dimensional latent representation called the bottleneck, while the decoder attempts to reconstruct the original input from this compressed representation. The bottleneck layer is the defining feature of an autoencoder — it forces the network to learn only the most essential and meaningful features of the data, discarding noise and redundancy in the process.

Objective and Functionality

The fundamental objective of a standard FCN is supervised learning — it is trained to map inputs to known target labels using labeled training data. For example, an FCN trained for customer churn prediction takes customer features as input and outputs a probability of churn. The model is optimized to minimize the difference between its predictions and the true labels. An autoencoder, by contrast, is primarily an unsupervised learning model. Its objective is not to predict an external label but rather to reconstruct its own input as accurately as possible after passing it through the compressed bottleneck. The reconstruction error — the difference between the original input and the reconstructed output — serves as the training signal. This means autoencoders can be trained on unlabeled data, making them extremely valuable in domains where labeled data is scarce or expensive to obtain.

Input-Output Relationship

In a standard FCN, the input and output are fundamentally different — the input is a feature vector and the output is a predicted class label or continuous value. In an autoencoder, the input and output are the same — or as close to the same as possible. The network learns to compress and then decompress the data, and the quality of the autoencoder is measured by how faithfully the reconstructed output matches the original input.

Simple Illustrative Example

Consider a dataset of 28×28 grayscale images of handwritten digits (such as the MNIST dataset). A standard FCN trained on this data takes the flattened 784-pixel vector as input and outputs one of ten class labels (digits 0–9). An autoencoder trained on the same data takes the same 784-pixel vector as input, compresses it down to perhaps 32 dimensions in the bottleneck layer, and then reconstructs the full 784-pixel image as output. The autoencoder never sees class labels — it simply learns the most compact and meaningful representation of the digit images through reconstruction.

Application of Autoencoders: Anomaly Detection

One of the most powerful and widely used applications of autoencoders is anomaly detection, particularly in cybersecurity and financial fraud detection. The core idea is elegant — an autoencoder is trained exclusively on normal, non-fraudulent transaction data. Through repeated training, it becomes highly proficient at reconstructing the patterns typical of legitimate transactions. When a fraudulent or anomalous transaction is then passed through the trained autoencoder, the network struggles to reconstruct it accurately because it has never encountered such patterns during training. The reconstruction error for this anomalous input is therefore significantly higher than for normal inputs. By setting a threshold on reconstruction error, the system can automatically flag transactions whose reconstruction error exceeds the threshold as potentially fraudulent, without ever having been explicitly trained on labeled fraud examples.

How Autoencoders Solve the Anomaly Detection Problem

Autoencoders solve the anomaly detection problem in a fundamentally unsupervised way, which is their greatest advantage in this context. In real-world fraud detection, labeled fraud examples are extremely

rare — fraudsters constantly evolve their techniques, meaning historical fraud labels quickly become outdated. A supervised FCN trained on labeled fraud data would struggle to detect new, previously unseen fraud patterns. An autoencoder, however, does not rely on fraud labels at all. It simply learns what normal looks like, and anything that deviates significantly from normal is flagged. This makes autoencoders highly adaptable and robust to novel attack patterns, giving them a significant practical advantage over traditional supervised approaches in dynamic, real-world environments.

Conclusion

In summary, while a standard FCN is a powerful supervised tool for classification and regression tasks, an autoencoder serves a fundamentally different purpose — learning compact, meaningful representations of data in an unsupervised manner. The encoder-decoder architecture and reconstruction-based training objective make autoencoders uniquely suited for tasks such as dimensionality reduction, anomaly detection, image denoising, and data generation, particularly in settings where labeled data is limited or unavailable.